



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Отчёт о лабораторной работе № 3
по курсу «Операционные системы»
«Список процессов в системе»

Студент: Ивенкова О. В.

Группа: ИУ9-42Б

Преподаватель: Брагин А. В.

Москва, 2025

Содержание

1	Постановка задачи	3
1.1	Часть 1. Вывод списка процессов из драйвера в ReactOS	3
1.2	Часть 2. Вывод списка процессов из драйвера в NetBSD	3
2	Практическая реализация	4
2.1	ReactOS	4
2.2	NetBSD	5
3	Результаты	7
3.1	ReactOS	7
3.2	NetBSD	7
4	Выводы	8
	Список литературы	8

1 Постановка задачи

1.1 Часть 1. Вывод списка процессов из драйвера в ReactOS

На основе драйвера из лабораторной работы № 2 создать новый драйвер, который выводит в отладочный лог список процессов в системе, используя API ядра. Список должен состоять из имени процесса (имя образа — Image Name), идентификатора процесса (PID) и идентификатора родительского процесса.

Драйвер должен загружаться и выгружаться по требованию так, чтобы можно было проверить соответствие списков процессов, выведенных драйвером, и с помощью утилиты Task Manager (taskmgr.exe).

Информацию о процессах получить из ядра с использованием функции `ZwQuerySystemInformation(SystemProcessInformation, ...)`. Выделение и освобождение памяти для информации о процессах реализовать, используя функции `ExAllocatePool(PagedPool, ...)` и `ExFreePool()`. Предусмотреть ситуацию, в которой запрошенный размер буфера, необходимый для размещения информации о процессах, может измениться в большую сторону между вызовами функции `ZwQuerySystemInformation`, и в этом случае произвести новое выделение памяти с освобождением старого буфера.

1.2 Часть 2. Вывод списка процессов из драйвера в NetBSD

На основе драйвера из лабораторной работы № 2 создать новый драйвер, который выводит в отладочный лог список процессов в системе, используя API ядра. Список должен состоять из имени процесса (имя образа — Image Name), идентификатора процесса (PID) и идентификатора родительского процесса.

Драйвер должен загружаться и выгружаться по требованию так, чтобы можно было проверить соответствие списков процессов, выведенных драйвером, и с помощью утилиты `ps`.

Информацию о процессах получить из ядра, используя глобальную переменную `allproc`. Для этого необходимо включить заголовочные файлы:

```
1 #include <sys/module.h>
2 #include <sys/kernel.h>
3 #include <sys/proc.h>
```

Самостоятельно следует изучить проблему защиты от «грязного чтения» списка процессов в момент его изменения операционной системой (например, при создании нового процесса или удалении завершенного).

2 Практическая реализация

2.1 ReactOS

- Создание модуля

- Перейти в дерево исходного кода ReactOS, полученного в результате выполнения лабораторной работы № 1. Зайти в папку `C:\Users\1\reactos\reactos\drivers`
- Создать в папке `drivers` новую папку с названием, например, `lab2`. (`mkdir lab2`)
- Добавить в конец файла `drivers\CMakeLists.txt` запись `add_subdirectory(lab2)`, для того чтобы папка `lab2` также участвовала в сборке системы
- В папку `drivers\lab2` поместить три файла:
 1. `CMakeLists.txt`, который будет информировать систему сборки о том, что именно и каким именно образом должно быть скомпилировано и линковано (так называемые правила сборки модуля);
 2. `lab2.c` — файл с исходным кодом драйвера;
 3. `lab2.rc` — файл с информацией о драйвере, который используется ReactOS и Windows
- `ninja bootcd`
- создать новую виртуальную машину, указав новый iso-образ
- В работающей системе в виртуальной машине выполнить загрузку драйвера по требованию с помощью следующих команд:

1. `sc create lab2driver binPath= C:\ReactOS\system32\drivers\lab2.sys
type= kernel`
 2. `sc start lab2driver` (запуск модуля, вывод фамилии в файл логов в PuTTY)
- `sc query type= driver` (вывести все драйвера)

Для ReactOS был разработан драйвер lab3, основанный на предыдущей версии lab2. Исходники скопированы и переименованы, обновлены пути в CMakeLists.txt.

Сборка выполнялась через `ninja`, драйвер переносился на виртуальную машину с помощью ISO-образа, созданного через `genisoimage`, и монтировался в `virt-manager`.

В коде использован вызов `ZwQuerySystemInformation` для получения списка процессов. Буфер выделяется динамически, при необходимости повторяется попытка. Для каждого процесса выводится PID, PPID и имя. Лог подписан фамилией и имя устройства персонализировано.

Загрузка драйвера выполнялась командой `sc create` и `sc start` в ReactOS. Вывод проверялся через отладочный лог.

2.2 NetBSD

- Создание нового загружаемого модуля ядра

- Создать файл `/usr/src/sys/dev/lab3.c`.
- Добавить минималистичную реализацию драйвера.
- Создать папку `lab3` в `/usr/src/sys/modules/`
- Создать файл `/usr/src/sys/modules/lab3/Makefile`.
- `cd /usr/src/sys/modules/lab3/`
- `make CC=/usr/bin/gcc`
- `modload ./lab3.kmod`

- Способы увидеть результат

- `dmesg` (просмотреть выведенные логи)
- `modunload lab3` (выгрузка модуля)

В NetBSD создан модуль `lab3` на основе `lab2`. Структура и названия файлов обновлены. Основная реализация в `lab3.c`.

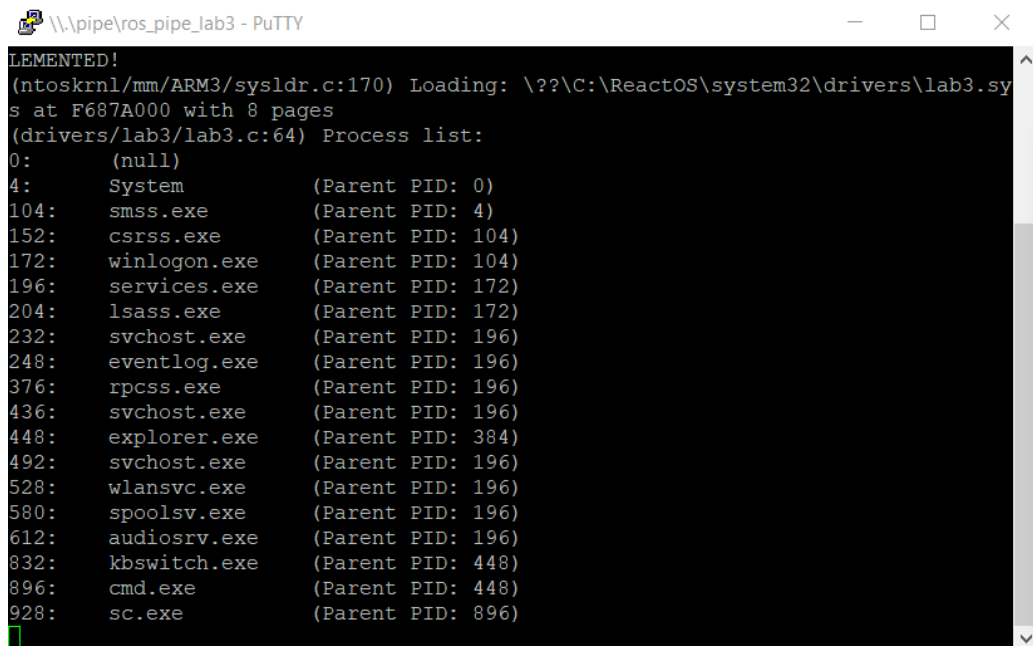
Использованы заголовки ядра: `proc.h`, `module.h`, `sysm.h` и другие. Основная функция — `list_processes()`, вызываемая из `modcmd`. Она обходит список `allproc` с защитой через `mutex_enter(proc_lock)`.

Выводятся `PID`, `PPID`, статус и имя команды каждого процесса. Сборка модуля — через `make` в `/usr/src/sys/modules/lab3`. Загрузка — через `modload`.

3 Результаты

3.1 ReactOS

Результат для ReactOS представлен на рисунке 1.



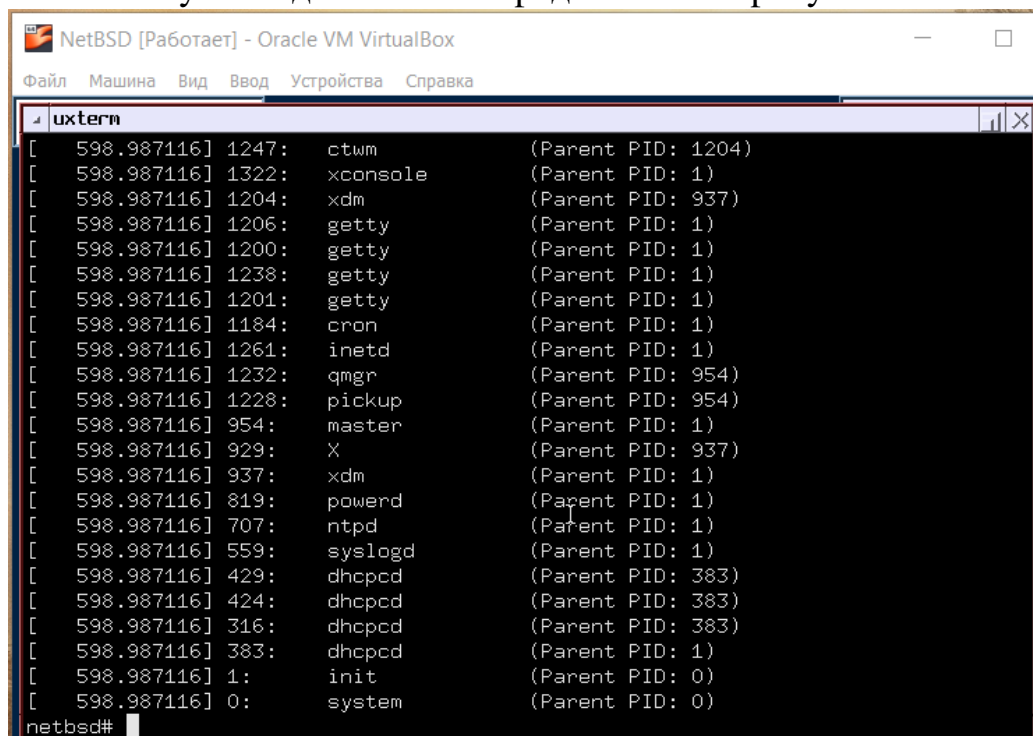
The screenshot shows a PuTTY window titled "\\pipe\ros_pipe_lab3 - PuTTY". The terminal output displays the process list for ReactOS. The text is as follows:

```
LEMENTED!  
(ntoskrnl/mm/ARM3/sysldr.c:170) Loading: \\??C:\ReactOS\system32\drivers\lab3.sys  
s at F687A000 with 8 pages  
(drivers/lab3/lab3.c:64) Process list:  
0:      (null)  
4:      System      (Parent PID: 0)  
104:    smss.exe     (Parent PID: 4)  
152:    csrss.exe    (Parent PID: 104)  
172:    winlogon.exe (Parent PID: 104)  
196:    services.exe (Parent PID: 172)  
204:    lsass.exe    (Parent PID: 172)  
232:    svchost.exe  (Parent PID: 196)  
248:    eventlog.exe (Parent PID: 196)  
376:    rpcss.exe    (Parent PID: 196)  
436:    svchost.exe  (Parent PID: 196)  
448:    explorer.exe (Parent PID: 384)  
492:    svchost.exe  (Parent PID: 196)  
528:    wlansvc.exe  (Parent PID: 196)  
580:    spoolsv.exe  (Parent PID: 196)  
612:    audiosrv.exe (Parent PID: 196)  
832:    kbswitch.exe (Parent PID: 448)  
896:    cmd.exe      (Parent PID: 448)  
928:    sc.exe       (Parent PID: 896)
```

Рис. 1 — Файл логов со списком процессов в ReactOS

3.2 NetBSD

Результат для NetBSD представлен на рисунке 2.



The screenshot shows a NetBSD terminal window titled "NetBSD [Работает] - Oracle VM VirtualBox". The terminal output displays the process list for NetBSD. The text is as follows:

```
uxtern  
[ 598.987116] 1247: ctwm      (Parent PID: 1204)  
[ 598.987116] 1322: xconsole (Parent PID: 1)  
[ 598.987116] 1204: xdm      (Parent PID: 937)  
[ 598.987116] 1206: getty    (Parent PID: 1)  
[ 598.987116] 1200: getty    (Parent PID: 1)  
[ 598.987116] 1238: getty    (Parent PID: 1)  
[ 598.987116] 1201: getty    (Parent PID: 1)  
[ 598.987116] 1184: cron     (Parent PID: 1)  
[ 598.987116] 1261: inetd    (Parent PID: 1)  
[ 598.987116] 1232: qmgr     (Parent PID: 954)  
[ 598.987116] 1228: pickup  (Parent PID: 954)  
[ 598.987116] 954: master  (Parent PID: 1)  
[ 598.987116] 929: X       (Parent PID: 937)  
[ 598.987116] 937: xdm      (Parent PID: 1)  
[ 598.987116] 819: powerd   (Parent PID: 1)  
[ 598.987116] 707: ntpd     (Parent PID: 1)  
[ 598.987116] 559: syslogd  (Parent PID: 1)  
[ 598.987116] 429: dhcpd    (Parent PID: 383)  
[ 598.987116] 424: dhcpd    (Parent PID: 383)  
[ 598.987116] 316: dhcpd    (Parent PID: 383)  
[ 598.987116] 383: dhcpd    (Parent PID: 1)  
[ 598.987116] 1:   init    (Parent PID: 0)  
[ 598.987116] 0:   system  (Parent PID: 0)  
netbsd#
```

Рис. 2 — Файл логов со списком процессов в NetBSD

4 Выводы

В рамках выполненной лабораторной работы была реализована задача по выводу списка активных процессов из драйвера операционной системы как в ReactOS, так и в NetBSD. Для обеих систем обеспечена корректная работа драйвера, который по запросу формирует список процессов и выводит его в отладочный лог. В список входят ключевые параметры каждого процесса: имя образа (Image Name), идентификатор процесса (PID) и идентификатор родительского процесса.

В случае с ReactOS сбор информации о процессах осуществлялся с использованием функции ядра ZwQuerySystemInformation с типом SystemProcessInformation. Были реализованы корректные механизмы выделения и освобождения памяти с использованием ExAllocatePool и ExFreePool, включая повторное выделение буфера в случае его недостаточного размера. Это позволило обеспечить устойчивость работы драйвера и избежать утечек памяти. Для NetBSD драйвер использовал доступ к глобальной переменной allproc с необходимыми включениями заголовочных файлов (<sys/module.h>, <sys/kernel.h>, <sys/proc.h>).

Обе реализации позволяют загружать и выгружать драйвер по требованию, что дало возможность сверить полученные списки с отображением в стандартных утилитах (taskmgr.exe для ReactOS и ps для NetBSD), подтвердив корректность работы.

Таким образом, поставленные задачи были успешно выполнены, а реализованные драйверы продемонстрировали корректное взаимодействие с ядром операционной системы и стабильную работу в предусмотренных сценариях.

Список литературы

- [1] А.В. Брагин, И.П. Иванов. *Операционные системы: Учебно-методическое пособие*. — М.: МГТУ им. Н.Э. Баумана, 2024. URL: <https://press.bmstu.ru/catalog/item/8226/reader/>
- [2] ReactOS Wiki. URL: <https://reactos.org/wiki/>
- [3] NetBSD Manual Pages. URL: <https://man.netbsd.org/>